

INFORME TÉCNICO DE GOBERNABILIDAD Y SEGURIDAD

Plataforma Big Data — Analítica Omnicanal de Ventas Retail

Fase 2 — Implementación y Optimización

Módulo: Gobernabilidad, Seguridad y Trazabilidad

Equipo	Ballerini · Torres · Vargas · Vásquez
Curso	Big Data y Cloud Computing — MDS UDD 2026
Docente	Prof. Luis Castillo Faune
Entrega	Fase 2 · 12 de junio de 2026
Notebook	04_Gobernabilidad.ipynb
Stack	GCP · BigQuery · PySpark · Great Expectations

Declaración IA: Este informe y el notebook asociado fueron asistidos por Claude (Anthropic) para estructuración y comentarios. Las decisiones arquitectónicas son propias del equipo.

1. Introducción y Contexto

El módulo de gobernabilidad y seguridad es el componente transversal del pipeline de Big Data que garantiza que los datos que fluyen desde las fuentes crudas hasta la capa Gold lo hagan con **trazabilidad, control de acceso y calidad documentada**. Su propósito no es solo cumplir un requisito académico, sino replicar las prácticas que un equipo de ingeniería de datos real aplicaría antes de exponer un Data Warehouse a stakeholders de negocio.

El enunciado de la Fase 2 exige explícitamente: (a) al menos un control de acceso implementado, (b) documentación del linaje de datos y (c) manejo explícito de datos sensibles. El notebook `04_Gobernabilidad.ipynb` cubre los tres puntos y los extiende con auditoría de ejecuciones y validación de calidad con Great Expectations.

1.1 Arquitectura del módulo

El módulo opera sobre Google Cloud Platform (GCP) en cinco pilares interconectados:

#	Pilar	Herramienta GCP	Tabla BigQuery creada
1	Control de Acceso (IAM)	BigQuery Dataset ACLs	—
2	Linaje de Datos	BigQuery Python SDK	retail_governance.data_lineage
3	Datos Sensibles	SHA-256 + Policy Tags	—
4	Auditoría de Pipeline	BigQuery Python SDK	retail_governance.pipeline_audit_log
5	Calidad de Datos	Great Expectations 0.18	retail_governance.data_quality_results

2. Configuración Inicial (Secciones 0 y 1)

2.1 Instalación de dependencias

El notebook instala cuatro paquetes sobre el entorno Google Colab usando `pip` con el flag `-q` para minimizar el output:

Paquete	Versión	Propósito
google-cloud-bigquery	última estable	SDK Python para interactuar con BigQuery (crear tablas, insertar filas, ejecutar queries)
google-cloud-storage	última estable	Acceso a Cloud Storage para referencias de rutas en el linaje
great_expectations	0.18.15	Framework de validación de calidad de datos. Versión fijada para reproducibilidad
db-dtypes	última estable	Tipos de datos BigQuery para Pandas (necesario para <code>to_dataframe()</code>)

2.2 Autenticación y parámetros globales

La autenticación usa `google.colab.auth.authenticate_user()`, que lanza el flujo OAuth2 de GCP en el navegador. Una vez completado, todas las llamadas al SDK de Google Cloud heredan automáticamente las credenciales del usuario autenticado.

Los parámetros globales centralizan la configuración en una sola celda, siguiendo el principio DRY (Don't Repeat Yourself):

Variable	Valor por defecto	Descripción
PROJECT_ID	retail-bigdata-udd	ID del proyecto GCP. Debe cambiarse al ID real del equipo
REGION	us-central1	Región donde se crean los datasets de BigQuery
BUCKET_NAME	data-lake-retail	Bucket GCS del Data Lake
DATASET_BQ	retail_gold	Dataset destino de los data marts analíticos
DATASET_GOV	retail_governance	Dataset de gobierno: tablas de linaje, auditoría y calidad

El dataset `retail_governance` se crea automáticamente si no existe. Actúa como el repositorio central de metadatos del pipeline.

3. Control de Acceso — IAM y Dataset ACLs (Sección 2)

3.1 Qué es y por qué importa

BigQuery implementa control de acceso a dos niveles: (a) IAM a nivel de proyecto, que gestiona quién puede crear o eliminar recursos, y (b) Dataset ACLs, que determinan quién puede leer o escribir datos dentro de un dataset específico. El notebook opera sobre el segundo nivel, que es el más granular y relevante para proteger datos de negocio.

El enunciado exige 'al menos un control de acceso implementado'. El modulo implementa cuatro roles distintos con sus respectivos niveles de acceso.

3.2 Roles configurados

Cuenta de servicio	Dataset	Rol BigQuery	Justificación
svc-dataeng@...iam	retail_gold + retail_governance	WRITER	Los Data Engineers construyen y actualizan todas las capas del pipeline
svc-analyst@...iam	retail_gold	READER	Los analistas solo consultan Gold; no deben modificar ni ver metadatos internos
svc-airflow@...iam	retail_gold	WRITER	Airflow/Cloud Composer necesita

Cuenta de servicio	Dataset	Rol BigQuery	Justificación
			escribir resultados de jobs orquestados
svc-auditor@...iam	retail_governance	READER	Los auditores solo leen logs y linaje; sin acceso a los datos de negocio

3.3 Cómo funciona la función `aplicar_politicas_acceso`

La función es idempotente: puede ejecutarse múltiples veces sin crear entradas duplicadas. El flujo es:

- Lee las ACLs actuales del dataset con `bq.get_dataset()`
- Extrae el conjunto de `entity_id` ya configurados
- Filtra las nuevas entradas descartando las que ya existen
- Actualiza el dataset solo con las entradas nuevas usando `bq.update_dataset()`

Idempotencia es crítica en pipelines reales donde el notebook puede re-ejecutarse sin efectos secundarios inesperados.

3.4 Verificación

La función `mostrar_politicas_dataset()` imprime un resumen tabular de todos los `AccessEntry` vigentes en un dataset, lo que permite auditar visualmente quién tiene acceso antes y después de aplicar cambios.

4. Linaje de Datos (Sección 3)

4.1 Concepto y enfoque adoptado

El linaje de datos documenta el origen, transformación y destino de cada campo a lo largo del pipeline. Responde tres preguntas fundamentales de auditoría:

- ¿De dónde viene este valor? (tabla_origen, capa_origen)
- ¿Qué transformación se le aplicó? (campo transformacion)
- ¿Es un dato sensible y cómo está protegido? (es_dato_sensible, tecnica_proteccion)

La estrategia adoptada es **registro manual estructurado en BigQuery**, válida según el enunciado que indica explícitamente que el linaje "puede ser manual". La alternativa (Cloud Data Catalog Lineage API) requiere Dataproc o Dataflow, herramientas que superan el presupuesto del free trial para este proyecto.

4.2 Esquema de la tabla data_lineage

Campo	Tipo	Descripción
id_linaje	STRING REQUIRED	UUID único generado con uuid.uuid4()
nombre_campo	STRING REQUIRED	Nombre de la columna documentada (ej: fecha_venta)
tabla_origen	STRING REQUIRED	Ruta GCS o tabla BQ de origen del campo
tabla_destino	STRING REQUIRED	Ruta GCS o tabla BQ destino
capa_origen	STRING REQUIRED	Capa Medallion: RAW BRONZE SILVER GOLD
capa_destino	STRING REQUIRED	Capa Medallion destino
transformacion	STRING NULLABLE	Descripción de la transformación aplicada al campo
notebook_origen	STRING NULLABLE	Nombre del notebook que ejecutó la transformación
es_dato_sensible	BOOL REQUIRED	TRUE si el campo contiene PII u otro dato sensible
tecnica_proteccion	STRING NULLABLE	HASH MASK SUPPRESS NONE
fecha_registro	TIMESTAMP REQUIRED	Timestamp de creación del registro
responsable	STRING REQUIRED	Cuenta de servicio que registró el linaje

4.3 Campos documentados por capa

El pipeline documenta 11 campos cubriendo las tres transiciones del Medallion:

Transición	Campos documentados	Transformaciones clave
RAW → Bronze	numero_transaccion, fecha_transaccion, venta, id_producto	Copia directa. Bronze preserva el dato crudo sin modificación
Bronze → Silver	fecha_venta, margen, margen_porcentaje, venta	Parseo de fecha (dd/MM/yyyy), cast a DOUBLE, campos derivados
Silver → Gold	venta_total, margen_total, numero_transacciones	Agregaciones SUM/COUNT por dimensiones de negocio

El linaje se registra con un timestamp y la cuenta de servicio responsable, lo que permite responder la pregunta '¿quién registró este linaje y cuándo?' en una auditoría.

5. Manejo de Datos Sensibles (Sección 4)

5.1 Contexto del dataset

El dataset de ventas retail actual (venta_tiendas.csv) no contiene PII en su forma presente: los campos disponibles son transaccionales (id_canal, id_producto, unidades, venta, costo) sin referencias a personas. Sin embargo, el módulo implementa los controles de forma preventiva y demostrativa para el escenario real donde el sistema se extienda con datos de clientes.

5.2 Las tres capas de protección

5.2.1 Pseudonimización con SHA-256

La función `pseudonimizar(valor, salt)` aplica el algoritmo SHA-256 con un salt fijo para transformar identificadores sensibles (RUT, email) en hashes de 64 caracteres.

Propiedades clave:

- Determinista: el mismo input produce siempre el mismo hash, lo que permite joins entre tablas
- No reversible: sin conocer el salt es computacionalmente inviable recuperar el valor original
- Compatible con BigQuery: el hash es un STRING estándar que puede indexarse y usarse en GROUP BY

Nota de producción: El salt debe cargarse desde *Google Secret Manager* y no estar hardcodeado en el código. La versión del notebook incluye el salt directamente solo para demostración académica.

5.2.2 Enmascaramiento a rangos categóricos

La función `asignar_rango_venta(monto)` reemplaza montos financieros exactos por rangos categóricos en la capa Gold. Los analistas de negocio ven 'DEVOLUCION_O_CERO', '0-10k', '10k-50k', '50k-100k' o '100k+', no los montos precisos por boleta.

Esto reduce el riesgo de correlación: con montos exactos un analista externo podría cruzar los datos con información pública para re-identificar transacciones de clientes conocidos.

5.2.3 Column-level Security con Policy Tags

BigQuery permite aplicar Policy Tags a columnas individuales. Un usuario sin el rol `datacatalog.categoryFineGrainedReader` recibe NULL al consultar esas columnas, aunque tenga acceso `READER` al dataset completo.

El notebook define tres niveles de clasificación:

Nivel	Columnas	Rol requerido
CONFIDENCIAL	numero_boleta, id_cliente_hash	roles/datacatalog.categoryFineGrainedReader
INTERNO	costo, margen_porcentaje	roles/datacatalog.categoryFineGrainedReader
PÚBLICO	fecha_venta, id_canal, id_producto, unidades, anio, mes	Sin restricción (READER del dataset)

La aplicación de Policy Tags requiere permisos de Data Catalog Admin. El notebook documenta el procedimiento paso a paso para la consola GCP dado que el free trial puede no tener esos permisos habilitados.

6. Registro de Auditoría del Pipeline (Sección 5)

6.1 Por qué auditar las ejecuciones

La auditoría del pipeline responde preguntas operacionales críticas: ¿cuándo se ejecutó el último procesamiento Gold? ¿Cuántos registros fueron descartados en Silver el martes pasado? ¿Qué cuenta de servicio ejecutó el pipeline a las 3 AM? Sin esta información, el equipo de operaciones no puede diagnosticar incidencias ni responder a preguntas de los stakeholders sobre la frescura y completitud de los datos.

6.2 Esquema de pipeline_audit_log

Campo	Tipo	Descripción
id_ejecucion	STRING	UUID único de la ejecución. Permite correlacionar el INICIO y el FIN de la misma corrida
etapa	STRING	BRONZE SILVER GOLD GOVERNANCE
notebook	STRING	Nombre del notebook ejecutado (trazabilidad de código)
estado	STRING	INICIO EXITO ERROR
registros_entrada	INTEGER	Cantidad de filas leídas desde la capa anterior
registros_salida	INTEGER	Cantidad de filas escritas en la capa actual
registros_descartados	INTEGER	Filas eliminadas por reglas de calidad (Silver) o agregación (Gold)
duracion_segundos	FLOAT64	Tiempo de ejecución de la etapa. Insumo para el análisis de rendimiento (Spark UI)
mensaje_error	STRING	Stack trace o mensaje de error si estado=ERROR
cuenta_ejecucion	STRING	Cuenta de servicio o usuario que disparó la ejecución
version_codigo	STRING	Git commit hash. Permite saber exactamente qué versión del código produjo cada resultado

6.3 La clase AuditorPipeline

La clase encapsula toda la lógica de registro de auditoría en dos métodos:

- `inicio()`: inserta un registro con estado INICIO al comenzar la etapa
- `fin(entrada, salida, descartados, duracion, error)`: inserta el registro de cierre con métricas

El mismo `id_ejecucion` aparece en ambos registros, lo que permite calcular la duración real y verificar si el INICIO siempre tuvo un FIN (detectar ejecuciones colgadas).

Para integrarla en los notebooks existentes solo se necesitan dos líneas adicionales:

```
auditor = AuditorPipeline('SILVER', '02_Capa_Silver.ipynb')
auditor.inicio()
# ... procesamiento Spark existente ...
auditor.fin(registros_entrada=12_000_000, registros_salida=11_987_340,
            registros_descartados=12_660, duracion_seg=45.7)
```

6.4 Métricas de la simulación

Etapa	Entrada	Salida	Descartados	% Descarte	Tiempo
BRONZE	12.000.000	12.000.000	0	0,000%	18,4 s
SILVER	12.000.000	11.987.340	12.660	0,106%	45,7 s
GOLD	11.987.340	3.650	0	0,000%	12,1 s
GOVERNANCE	0	0	0	—	3,2 s

El 0,106% de descarte en Silver corresponde a registros con fecha inválida o id_producto nulo, consistente con los filtros aplicados en 02_Capa_Silver.ipynb.

7. Validación de Calidad con Great Expectations (Sección 6)

7.1 Concepto de contrato de calidad

Great Expectations permite definir expectativas (assertions) sobre los datos que actúan como un contrato formal entre productores y consumidores. Si los datos de Silver no cumplen las expectativas, el pipeline debe detenerse antes de promover datos corruptos a Gold.

7.2 Las 7 expectativas del contrato

Expectativa	Columna	Regla	Justificación de negocio
id_producto_no_nulo	id_producto	not_be_null	Todo registro Silver debe tener producto identificado para KPIs de rotación
id_canal_valores_validos	id_canal	in_set([1, 2, 3])	Solo existen 3 canales: Tienda, E-commerce, Mayorista. Valores fuera del set indican corrupción
anio_rango_valido	anio	between(2019, 2026)	El dataset histórico parte en 2019. Un año anterior indica error de parseo de fecha
mes_rango_valido	mes	between(1, 12)	El mes debe ser válido. Valores fuera del rango indican error de extracción
venta_no_nula	venta	not_be_null	Toda transacción debe tener monto. Un NULL en venta invalida los KPIs de ingresos
transacciones_unicas	numero_transaccion	to_be_unique	Los duplicados inflan artificialmente los conteos de ventas
fecha_venta_no_nula	fecha_venta	not_be_null	Sin fecha no es posible calcular tendencias temporales ni KPIs mensuales

7.3 Persistencia de resultados

Los resultados de cada validación se guardan en `retail_governance.data_quality_results` con timestamp, cantidad de registros fallidos y porcentaje de fallo. Esto permite:

- Tendencias de calidad: ¿está mejorando o empeorando la calidad de los datos de origen?
- SLA de datos: definir umbrales máximos de fallo (ej: no más del 0,5% de fechas inválidas)
- Dashboards de calidad: conectar la tabla a Looker Studio para monitoreo continuo

7.4 Integración recomendada con Airflow

En el DAG de Airflow, la tarea de validación GE debe ejecutarse como paso previo a la tarea Gold:

```
validar_silver >> escribir_gold
```

Si la validación falla (alguna expectativa no se aprueba), Airflow marca la tarea como FAILED y la tarea `escribir_gold` nunca se ejecuta, protegiendo la capa analítica de datos de baja calidad.

8. Resumen Ejecutivo y Próximos Pasos

8.1 Artefactos producidos por el módulo

Artefacto	Tipo	Propósito
04_Gobernabilidad.ipynb	Notebook Colab	Implementación ejecutable de los 5 pilares de gobierno
retail_governance.data_lineage	Tabla BigQuery	Linaje campo a campo: 11 registros RAW→GOLD documentados
retail_governance.pipeline_audit_log	Tabla BigQuery	Log de auditoría por ejecución del pipeline
retail_governance.data_quality_results	Tabla BigQuery	Historial de validaciones Great Expectations por run
Clase AuditorPipeline	Módulo Python	Reutilizable en los notebooks 01, 02 y 03 con 2 líneas de código
función pseudonimizar()	Función Python	SHA-256 + salt para identificadores sensibles
función asignar_rango_venta()	Función Python	Enmascaramiento de montos a rangos categóricos
CLASIFICACION_COLUMNS	Diccionario Python	Taxonomía de Policy Tags con 3 niveles de sensibilidad

8.2 Cobertura de los requisitos del enunciado

Requisito Fase 2	Estado	Implementación
Al menos un control de acceso implementado	Cumplido	4 roles IAM con Dataset ACLs en retail_gold y retail_governance
Documentación del linaje de datos (puede ser manual)	Cumplido	11 campos documentados en retail_governance.data_lineage
Manejo explícito de datos sensibles si el caso lo requiere	Cumplido	SHA-256, enmascaramiento y taxonomía de Policy Tags documentada

8.3 Próximos pasos recomendados

- Aplicar Policy Tags en la consola GCP: BigQuery Studio → Dataset → Tabla → Schema → columna → Edit → Policy tag
- Habilitar Cloud Audit Logs en: IAM & Admin → Audit Logs → BigQuery → Data Read + Data Write
- Integrar AuditorPipeline.inicio() y .fin() en los notebooks 01, 02 y 03 como primeras y últimas celdas
- Agregar la validación Great Expectations como tarea prereq en el DAG de Airflow antes de la escritura Gold
- Conectar retail_governance.data_quality_results a Looker Studio para un dashboard de calidad en tiempo real

El dataset retail_governance actúa como Data Catalog interno del proyecto. En una arquitectura productiva, estas tablas se complementarían con los metadatos automáticos de Cloud Data Catalog y Dataplex.